

Assignment 2 Design Note

Sachi Thonse Rao (2023101120), Sama Harshika (2023101004)

1. What We Built

1.1 Pipeline overview

The system is a two-stage retrieve-then-rank pipeline built on top of Assignment 1's lexical (BM25) and semantic (sentence-embedding, paraphrase-multilingual-MiniLM-L12-v2, 384-d) candidate generators, evaluated on both MIND and EB-NeRD.

Stage 1 -- Retrieval. For each impression, both a BM25 score and a semantic (cosine similarity) score are computed for every candidate against a query built from the user's point-in-time click history. The top-K (K=150) candidates by the primary method (semantic, by default) are kept; for most impressions this is a no-op since the official candidate lists are already smaller than K, but real for the long tail (some MIND impressions carry 400+ candidates).

Stage 2 -- Re-rank. A LightGBM LambdaMART model scores the retained candidates. Three versions exist, each evaluated against the previous one on the same data:

- **v1 (original Q2 model, 14 features):** the two retrieval scores plus 12 behavioural features (Table 1, top four rows). This is the model behind the Codabench submissions in Section 6.
- **Improved offline model (44 features):** adds short-term popularity, exposure-age and within-impression relative features, drops the two position features, and is early-stopped (Sections 1.2-1.4, results in 2.1).
- **v2 MIND-large model (36 features):** trained on labelled MINDlarge_dev with one feature function shared by training and test-time scoring, using only features that exist on the unlabelled test set (Section 2.4).

1.2 Behavioural features (Part I.1)

Group	Features	Used by
Click history	n_clicks_before, recency_weighted_click_count (exp. decay, 7-day half-life), is_cold_start, days_since_last_click	v1 (first 3), improved
Article	popularity_prior_ctr (strictly-prior cumulative CTR), freshness_hours, category_match, category_match_frac	v1, improved
Session	session_click_count_before, session_impressions_before, avg_dwell_time_before	v1, improved
Position	position, position_ctr_prior	v1 only (ablation in 2.2)
Short-term popularity (new)	clicks / impressions / CTR in the strictly-prior 1h and 24h windows; cum_clicks_prior, cum_impressions_prior	improved

Group	Features	Used by
Exposure age (new)	hours_since_first_seen (time since the article was first shown to anyone; a publish-time proxy for MIND, which has none)	improved, v2
Within-impression (new)	percentile rank, z-score and gap-to-best of the semantic, BM25, popularity, exposure and age scores inside each impression; n_candidates	improved, v2

Table 1: Behavioural features. LambdaMART only compares candidates within one impression, so the relative features ('best semantic match in this impression') carry more signal than raw scores whose scale drifts from user to user.

1.3 Point-in-time correctness and two data fixes

Every feature for impression I uses only events strictly before I's own timestamp; same-timestamp candidates are bucketed so they never see each other's click label, and the 1h/24h windows cover [t-W, t). The boundary is enforced by an `assert_no_leakage` check on every build and by 21 synthetic unit tests (same-timestamp exclusion, strictly-prior windows, cold-start handling, freshness never negative, an in-window click never visible to its own impression).

Re-running the MIND pipeline end to end exposed two data bugs that affected every earlier MIND number:

- **Impression-ID collision.** MIND numbers impressions from 1 in each `behaviors.tsv`, so all 73,152 dev impressions (the entire test split) shared an ID with an unrelated train/val impression and inherited that user's history, session features and Stage-1 query. Fixed by prefixing IDs with their source file; a new assertion fails the build if an ID ever covers two users or timestamps.
- **History timestamps.** The parser stamped every history item with the impression's own time and repeated the history per impression, so a user's first impression saw an empty history and later ones counted duplicates. History is now de-duplicated, pre-window clicks are stamped before the window, and in-window clicks are added at their real time. Impressions with an empty history fell from 33.4% to 0.6%; the cold-start share fell from 38.3% to 13.4%.

EB-NeRD ships `session_id` and `read_time` natively; MIND has neither, so sessions are derived from a 30-minute inactivity gap and dwell time is left NaN (documented, not silently imputed).

1.4 Re-ranker architecture and why LightGBM

We chose Option A (GBDT) over a neural ranker: LightGBM's LambdaMART directly optimizes an NDCG-like ranking objective, trains in seconds-to-minutes on this feature set, handles missing values (e.g. MIND's absent freshness) natively, and required no additional infrastructure beyond what Part I.1 produced. Each impression is one query group over its full retained candidate list. v1 uses a fixed 200-tree configuration; the improved and v2 models use up to 2,000 trees (learning rate 0.03, 63 leaves, row/column subsampling 0.8) with early stopping on `nDCG@10` over a held-out 10% of training impressions -- never the val/test data that metrics are reported on.

1.5 Baseline reproduction: NRMS (Part I.3)

Separately, we reproduced NRMS (Wu et al., 2019): a news encoder (word embedding -> multi-head self-attention over title tokens -> additive attention pooling) and a user encoder (the identical self-attention/additive-attention pattern applied over the sequence of the user's recent news vectors), scored by a dot product between user and candidate vectors. Trained with pointwise BCE over 1 positive + 4 sampled negatives per impression -- NRMS's defining contribution is its two encoders, not its training loss, and this lets both the baseline and improved model reuse the same per-impression AUC/MRR/nDCG harness as the GBDT re-ranker.

The one principled improvement (Part I.3.2): a learned category embedding, concatenated onto the title-attention output inside the news encoder and projected back to the same width. This is the only architectural difference between baseline and improved models -- the user encoder, scoring function, training data, hyperparameters, and random seed are held identical -- so the ablation isolates exactly one change.

2. Baseline vs. Improved Results

2.1 Re-ranker: before vs. after (Part I.2)

Table 2 reports AUC/MRR/nDCG@5/nDCG@10 before re-ranking (Stage 1's semantic score alone) and after, for v1 and the improved model trained on the same sample (MIND-small with both data fixes applied; 20,000 train and 5,000 val / 5,000 test impressions).

Split	Model	AUC	MRR	nDCG@5	nDCG@10
val	before (semantic)	0.589	0.292	0.271	0.332
val	after, v1 (14 feat.)	0.685	0.367	0.338	0.401
val	after, improved (44)	0.708	0.388	0.361	0.422
test	before (semantic)	0.610	0.328	0.309	0.371
test	after, v1 (14 feat.)	0.690	0.400	0.382	0.440
test	after, improved (44)	0.696	0.407	0.389	0.447

Table 2: MIND. Re-ranking lifts every metric on both splits (test AUC 0.610 -> 0.696). Before the data fixes the same pipeline scored test AUC 0.525 before / 0.639 after; v1 itself rises to 0.690 on the corrected features, so most of the test-split gain since the first version comes from the fixes, and the improved feature set adds a further significant +0.006 (Table 3).

For EB-NeRD (demo) the original v1 run is unchanged: AUC 0.532 -> 0.724 on val and 0.534 -> 0.749 on test; nDCG@10 0.448 -> 0.587 (val) and 0.448 -> 0.618 (test). EB-NeRD was not affected by either data bug (its impression IDs are unique and its history carries real timestamps); the improved feature set has so far only been run on MIND.

2.2 Ablation: improved vs. v1, and features unavailable at serving time (Q9)

Both deltas below are paired bootstraps over per-impression differences on identical impressions; * = 95% CI excludes zero.

Comparison	Split	dAUC	dMRR	dnDCG@5	dnDCG@10
improved - v1	val	+0.023*	+0.021*	+0.024*	+0.022*
improved - v1	test	+0.006*	+0.007	+0.007	+0.006*
(+position) - improved	val	-0.003*	-0.003	-0.003	-0.003
(+position) - improved	test	+0.001	+0.000	-0.001	+0.000

Table 3: The improvement is significant on all four metrics on val and on AUC and nDCG@10 on test (MRR and nDCG@5 CIs touch zero at $n=4,993$). Test CIs: dAUC [+0.0006, +0.0107], dnDCG@10 [+0.0004, +0.0116].

The position feature is where the candidate sat in the dataset's logged list -- decided by the production system that collected the log, so a fresh request cannot have it before ranking. Adding it (and position_ctr_prior) back to the improved model gives no gain on test and a small significant loss in val AUC, so the serving-safe model loses nothing by excluding it. The largest-gain features of the improved model are the within-impression z-score of popularity CTR, the 1-hour CTR, the semantic score, cumulative CTR and the 24-hour CTR z-score: short-term popularity is the main new signal.

2.3 NRMS baseline vs. category-aware improvement, with paired bootstrap CI (Part I.3.3-4)

Both models were trained at 8,000 train impressions, 8 epochs, 3,000 eval impressions. Significance is a paired bootstrap over per-impression (improved - baseline) deltas; only deltas whose 95% CI excludes zero are reported as significant.

Dataset	Split	dAUC	dMRR	dnDCG@5	dnDCG@10
EB-NeRD	val	+0.0182*	+0.0174*	+0.0173*	+0.0154*
EB-NeRD	test	+0.0257*	+0.0204*	+0.0225*	+0.0204*
MIND	val	+0.0367*	+0.0278*	+0.0274*	+0.0277*
MIND	test	-0.0108*	-0.0103*	-0.0078*	-0.0082*

Table 4: * = 95% CI excludes zero. EB-NeRD: consistently significant and positive on both splits. MIND: significant and positive on val, but significantly negative on test.

These NRMS runs predate the two MIND data fixes in Section 1.3. The MIND test reversal reproduced across two runs at two scales, so it is not sampling noise; but every MIND test impression had been fed a different user's click history, which gives a concrete explanation that is more specific than the date-shift hypothesis we first proposed. We therefore claim the category-aware gain for EB-NeRD (significant on both splits) and treat MIND as unresolved until NRMS is re-run on the corrected features.

2.4 MIND-large v2 re-ranker (the model behind the updated submission)

v1 was trained on MIND-small features but scored MINDlarge_test with features computed differently at submission time (history capped at 10 items, unseen-article popularity 0 instead of missing, a different meaning of position). v2 removes that train/serve mismatch: one feature function (src/mind_large_v2.py) is used for both training and test-time scoring, and it only uses signals that exist on the unlabelled test set:

- retrieval: BM25 and semantic score over the last 10 history items, plus a longer 50-item semantic profile;
- user: full history length and the category/subcategory share of the history;
- article priors: clicks, impressions and CTR from earlier labelled files only (train when training on dev; train+dev when scoring test);
- exposure: how often the article was shown in the previous 1/6/24 full hours across all behaviors files up to the one being scored (exposure counts only, never clicks), and hours since first shown;
- within-impression relative versions of the above, and the candidate count.

Training: 50,000 sampled MINDlarge_dev impressions (1,880,266 candidate rows) split 75% fit / 10% early stopping / 15% holdout; priors from 27,046 articles with labelled train history; 499,071 (article, hour) exposure buckets; early stopping chose 709 trees. On the 7,500-impression holdout, the currently-submitted v1 model was scored on its own 14 features, computed exactly as its submission script computes them:

Model	AUC	MRR	nDCG@5	nDCG@10
semantic only	0.625	0.336	0.315	0.376
v1 (submitted)	0.660	0.368	0.351	0.410
v2	0.758	0.467	0.444	0.499
v2 - v1 (paired)	+0.097* [0.091, 0.103]	+0.099* [0.092, 0.107]	+0.093* [0.086, 0.100]	+0.089* [0.083, 0.095]

Table 5: MINDlarge_dev holdout (7,500 impressions), 95% CIs in brackets. v2 beats the submitted model on every metric by about +0.09-0.10. The holdout is the day before the test week, so the leaderboard gain may be smaller.

The top v2 features by gain are subcategory share of the history, prior CTR, candidate count, and the within-impression gap/z-score of the 50-item semantic score -- i.e. a longer user profile and finer-grained topic match matter most once features are computed consistently. Scoring the full MINDlarge_test (2,370,727 impressions) took 74.6 minutes in one process (530 impressions/s), streaming predictions straight into the zip.

3. Serving and Scale Analysis (Part I.4)

Dataset	Index+store memory	p50	p99	Stage split (retr/feat/rerank)	Cost/1k queries
MIND	162.6 MB	~24 ms	~51 ms	36% / 24% / 40%	\$0.0004

Dataset	Index+store memory	p50	p99	Stage split (retr/feat/rerank)	Cost/1k queries
EB-NeRD	25.5 MB	~20 ms	~33 ms	14% / 43% / 43%	\$0.0003

Table 6: Single-request benchmark of the v1 pipeline (2,000 requests, 2 runs averaged). Both meet a 100ms p99 SLA on a single core at negligible cost. Index memory matches theory (MIND: 65,238 articles x 384 dims x 4 bytes ~ = 95.6MB semantic index).

The stage-time split is the most instructive measured result. Retrieval's share tracks catalog size directly -- 36% for MIND's 65,238-article catalog vs. 14% for EB-NeRD's 11,777-article catalog -- because search over a brute-force flat index is $O(\text{catalog size})$. Feature-assembly and rerank are both $O(K)$ (candidates retrieved, fixed at 150 for both datasets), so their absolute cost stays roughly flat across the two very differently-sized catalogs. The stage that is largest today is therefore not necessarily the stage that will hurt most at scale -- retrieval is the only one whose cost is provably tied to catalog size.

The later models change two numbers. The semantic index for the MINDlarge corpus (130,379 articles) is 193 MB (130,379 x 384 x 4 bytes), double MIND-small's. The model grows from 704 KB (v1, 200 trees) to 1.6 MB (improved, 237 trees) and 4.9 MB (v2, 709 trees); tree count is what drives predict() cost, so v2's rerank stage is the one to re-benchmark. Batch throughput for v2, measured on the real test set, is 530 impressions/s (~37 candidates each) including feature computation, in one process.

At 10x catalog size, the fix ahead of that point is swapping brute-force IndexFlatIP search for an approximate index (FAISS IVF/HNSW), trading a small amount of recall for sub-linear search time. At 10x query volume the first thing that breaks is horizontal capacity: the serving path is a single Python process that serializes requests (no free parallelism from the GIL on this CPU-bound path), so 10x QPS needs roughly 10x more replicas, and beyond a handful of replicas the in-process feature stores (34MB for MIND) must move to a shared store (Redis/DynamoDB-style). The new exposure features make this more pressing: 1h/6h/24h counts change every hour, so they need a streaming counter service rather than a periodically refreshed snapshot.

4. Feature Store and Re-ranker Anatomy (Assignment 2, Click-Log-Derived)

This section covers what is new in Assignment 2: the click-log-derived feature store (Part I.1) and the LightGBM re-rankers (Part I.2). Offline sizes are for MIND after the data fixes.

4.1 Anatomy, with sizes

Artifact	Size / shape
behavioral_features.parquet (offline, MIND)	8,584,442 rows (1 per impression x candidate), 30 columns, 287 MB on disk
click_history_features.parquet (offline, MIND)	230,117 rows (1 per impression -- 73,152 more than before the ID fix), 390 MB (incl. 384-d history embeddings)

Artifact	Size / shape
session_features.parquet (offline, MIND)	230,117 rows, 5.5 MB
user_features.parquet (offline, MIND)	45,214 rows (1 per user), 8.3 MB
Re-ranker model files	v1 704 KB (200 trees, 14 features); improved 1.6 MB (237 trees, 44 features); v2 4.9 MB (709 trees, 36 features)
MINDlarge semantic embeddings	130,379 x 384 float32, 193 MB (one-time build: 1h30m on CPU)
article_store / user_store (online, in-RAM)	10.83 MB / 23.14 MB (measured with v1, Section 3)

Table 7: Feature-store anatomy. The retrieval scores are computed at Stage-1 scoring time, not stored offline. The v2 pipeline does not use this offline table at all: it streams behaviors files and builds features per 10,000-impression chunk, so its memory does not grow with the test-set size.

4.2 Latency and throughput

v1 (measured before the updates): training on 5,000 sampled impressions (176,430 candidate rows) took 2.62s -- 67,463 rows/sec. Inference (predict() only, isolated from feature assembly): 500 requests, 36.9 candidates/request on average -- mean 1.483ms, p50 0.688ms, p99 14.843ms; Little's Law $X \cdot R_{\text{bar}} = 1.0000$.

Updated pipeline (measured): rebuilding the full MIND behavioural feature table (8.58M candidate rows, including the new windowed counts, vectorised with prefix sums and np.searchsorted rather than per-article loops) takes 2:02 at 5.5 GB peak memory; the improved re-ranker run (Stage 1 scoring, three models, evaluation) takes 2:07 at 5.0 GB. v2 training (feature building for 50,000 impressions at ~490 impressions/s plus LightGBM) and full test scoring at 530 impressions/s are reported in Section 2.4.

5. Extended Evaluation (Part I.5)

The full two-stage pipeline with the improved re-ranker was evaluated on MIND (corrected data) with the complete metric suite -- AUC, MRR, nDCG@5, nDCG@10, intra-list diversity, novelty and catalog coverage -- each with bootstrap 95% CI, on 2,996 scored impressions per split, sliced by cold-start vs. warm and head vs. tail (head = the clicked article is at or above the 80th percentile of train-split popularity).

Split / slice	n	AUC	MRR	nDCG@10	Diversity	Novelty
val overall	2,996	0.703	0.385	0.418	0.923	16.19
val cold-start	592	0.700	0.373	0.424	0.923	16.22
val warm	2,404	0.704	0.388	0.416	0.923	16.18
val head	896	0.716	0.329	0.353	0.915	15.13
val tail	2,100	0.698	0.408	0.446	0.926	16.64
test overall	2,996	0.693	0.406	0.444	0.918	17.22
test cold-start	799	0.682	0.400	0.459	0.924	17.14

Split / slice	n	AUC	MRR	nDCG@10	Diversity	Novelty
test warm	2,197	0.697	0.408	0.439	0.915	17.25
test head	313	0.588	0.366	0.402	0.928	16.68
test tail	2,683	0.705	0.410	0.449	0.917	17.29

Table 8: MIND extended evaluation, improved re-ranker. Catalog coverage: 0.0244 (val) and 0.0182 (test) of 65,238 articles. Previously (before the fixes, v1): val AUC 0.676 / nDCG@10 0.329, test AUC 0.692 / nDCG@10 0.307.

Compared with the earlier run, nDCG@10 rises from 0.329 to 0.418 on val and from 0.307 to 0.444 on test. Cold-start and warm users now perform comparably on both splits (AUC gaps of 0.004 and 0.015), so the behavioural features do not systematically fail new users. The test head slice is the clear weak spot: only 313 of 2,996 test clicks go to train-popular articles, and there AUC drops to 0.588 -- on the test day most clicked articles are new, and 'popular in train' articles are already stale, which is exactly the regime the short-term popularity features were added for. Catalog coverage is low in absolute terms because 2,996 impressions x 10 recommendations can touch at most a few percent of a 65K catalog; intra-list diversity stays high (0.92).

EB-NeRD was evaluated with the same harness (full two-stage pipeline with the v1 re-ranker, bootstrap 95% CI on every metric, same cold-start/warm slicing):

Dataset	Metric	Overall	Cold-start	Warm
EB-NeRD	AUC	0.768	0.745	0.768
EB-NeRD	Diversity	0.856	0.857	0.856
EB-NeRD	Coverage	0.041 (of 11,777 articles)		

Table 9: EB-NeRD extended evaluation (val split, n=300 sampled impressions, v1 re-ranker). Cold-start and warm users perform comparably -- the behavioural features do not systematically fail new users.

EB-NeRD's catalog coverage (4.1%) is higher than MIND's because its catalog is about 5.5x smaller (11,777 vs. 65,238 articles), so the same number of top-10 lists covers a larger share of it; the low absolute value reflects the n=300 evaluation sample relative to catalog size, not a structural diversity failure -- intra-list diversity stays high (0.856). EB-NeRD's overall AUC (0.768) is also higher than MIND's (0.693-0.703), consistent with EB-NeRD's richer native session and dwell-time signal.

6. Codabench Submission

Both leaderboard submissions used the full two-stage pipeline (BM25/semantic retrieval -> LightGBM re-rank), scored on the official large test sets (MINDlarge_test, ebnerd_testset) via the generate*_reranked_submission.py scripts.

Competition	Submission	Score	Status	ID
MIND News Recommendation	submission_reranked.zip	0.6128	Finished	924797
Ekstra Bladet (EB-NeRD)	submission_reranked.zip	0.6649	Finished	925011

ID # ▾	File name	Date	Status	Score	Actions
924797	submission_reranked.zip	2026-09-13 17:08	Finished	0.6128	✓ ↗

ID # ▾	File name	Date	Status	Score	Detailed Results	Actions
925011	submission_reranked.zip	2026-09-13 21:49	Finished	0.6649	👁	✓ ↗

Two engineering caveats are worth noting for reproducibility: the re-ranker was trained on small/demo-scale features and applied to large-scale test data (a documented train/serving distribution shift -- feature semantics match, scale does not exactly); MIND's raw test history has no per-item timestamps, so `recency_weighted_click_count` falls back to a simple click count there (EB-NeRD's history does carry timestamps, so real recency-weighting and dwell time work as designed).

Github Link: <https://github.com/harshika1012/IRE-A1P2>

7. Where the System Breaks at 10x Scale

- **10x catalog size:** the brute-force semantic index is the structural bottleneck -- its cost is the only one in the pipeline that is $O(\text{catalog size})$. Fix: FAISS IVF/HNSW in place of IndexFlatIP. MINDlarge already doubles the index (193 MB for 130K articles); a 10x catalog (~1.3M articles) is ~2 GB, still RAM-resident -- it is search time, not memory, that degrades first for retrieval.
- **10x query volume:** a single serving process breaks first, well before the model or index -- GIL-bound Python serializes requests, so throughput scales only with replica count. Fix: horizontal scaling, plus moving feature stores (and the hourly exposure counters the new features need) out of each replica into a shared store.
- **10x training data (Part I.1/I.2):** the offline pipeline materialises the full behavioural table (8.6M rows, 5.5 GB peak for MIND-small); a 10x larger table would not fit. The v2 path shows the fix: stream the behaviors files, keep only compact aggregates (article priors, 499K hourly exposure buckets) in memory, and build features per chunk -- 2.37M test impressions were scored this way in one process.
- **10x model size:** v2 already uses 709 trees vs. v1's 200; `predict()` cost grows linearly with trees, so at 10x traffic the rerank stage needs batching across concurrent requests or a distilled/shallower model.
- **10x NRMS training data:** tractable on CPU at the scales tested (8,000 train impressions, ~15-17 minutes/dataset), but word-level self-attention training does not trivially parallelize across a 10x larger vocabulary and history length without GPU acceleration or reduced precision.